

MODUL PRAKTIKUM ALGORITMA DAN PEMROGRAMAN 2 (PYTHON)

Yusri Ikhwani, S.Kom., M.Kom
Teknik Informatika dan Sistem Informasi

Menggunakan Visual Studio Code

Python 3

"Code is like humor. When you have to explain it, it's bad."

— Cory House

DAFTAR ISI

BAB 1	Percabangan (Selection)	1
BAB 2	Perulangan (Looping)	9
BAB 3	Struktur Data Dasar	17
BAB 4	Fungsi (Function)	26
BAB 5	Algoritma dan Logika Pemrograman	34
BAB 6	Pengolahan File Sederhana	42
BAB 7	Debugging dan Error Handling	50
BAB 8	Mini Project (Studi Kasus)	58

BAB 1 PERCABANGAN (SELECTION)

Tujuan Praktek:

- ◆ Memahami konsep percabangan bersyarat dalam bahasa Python
- ◆ Mampu mengimplementasikan pernyataan *if*, *if-else*, *if-elif-else*, dan *nested if*
- ◆ Mampu menyelesaikan masalah yang memerlukan logika pengambilan keputusan
- ◆ Memahami penggunaan operator relasi dan operator logika dalam percabangan

1.1 Dasar Teori

Percabangan (selection) adalah struktur kontrol yang memungkinkan program mengambil keputusan berdasarkan kondisi tertentu. Dalam Python, percabangan diimplementasikan menggunakan kata kunci *if*, *elif*, dan *else*. Struktur ini sangat penting karena memberikan kemampuan pada program untuk bercabang sesuai logika yang diinginkan pemrogram.

Bentuk umum percabangan *if-elif-else* adalah sebagai berikut:

```
1 if kondisi_1:
2     # blok kode jika kondisi_1 bernilai True
3 elif kondisi_2:
4     # blok kode jika kondisi_2 bernilai True
5 else:
6     # blok kode jika semua kondisi di atas bernilai False
```

Operator relasi yang sering digunakan: *==* (sama dengan), *!=* (tidak sama), *>* (lebih besar), *<* (lebih kecil), *>=* (lebih besar sama dengan), *<=* (lebih kecil sama dengan). Sedangkan operator logika meliputi *and*, *or*, dan *not*.

1.2 Latihan Praktikum

Latihan 1.1 — Menentukan Kelulusan Mahasiswa

Buatlah program yang menentukan status kelulusan mahasiswa berdasarkan nilai akhir. Jika nilai *>= 60* maka dinyatakan LULUS, selain itu TIDAK LULUS.

 **Nama File:** `latihan1_1_kelulusan.py`

Kode Program:

```
1 print("==== Program Cek Kelulusan =====")
2 nama = input("Masukkan nama mahasiswa : ")
3 nilai = float(input("Masukkan nilai akhir : "))
4
5 if nilai >= 60:
6     status = "LULUS"
```

```

7  else:
8      status = "TIDAK LULUS"
9
10 print("-" * 35)
11 print(f>Nama : {nama}")
12 print(f>Nilai : {nilai}")
13 print(f>Status: {status}")

```

Hasil Eksekusi Program:

```

===== Program Cek Kelulusan =====
Masukkan nama mahasiswa : Andi Saputra
Masukkan nilai akhir : 75
-----
Nama : Andi Saputra
Nilai : 75.0
Status: LULUS

```

Latihan 1.2 — Konversi Nilai Angka ke Huruf

Buatlah program yang mengkonversi nilai angka menjadi nilai huruf dengan ketentuan: A (≥ 85), B (70-84), C (55-69), D (40-54), E (< 40).

 **Nama File:** latihan1_2_nilaihuruf.py

Kode Program:

```

1  print("===== Konversi Nilai Angka ke Huruf =====")
2  nilai = float(input("Masukkan nilai angka (0-100): "))
3
4  if nilai >= 85:
5      huruf = "A"
6      ket = "Sangat Baik"
7  elif nilai >= 70:
8      huruf = "B"
9      ket = "Baik"
10 elif nilai >= 55:
11     huruf = "C"
12     ket = "Cukup"
13 elif nilai >= 40:
14     huruf = "D"
15     ket = "Kurang"
16 else:
17     huruf = "E"
18     ket = "Sangat Kurang"
19
20 print(f">Nilai Angka : {nilai}")
21 print(f">Nilai Huruf : {huruf}")
22 print(f">Keterangan : {ket}")

```

Hasil Eksekusi Program:

```

===== Konversi Nilai Angka ke Huruf =====
Masukkan nilai angka (0-100): 78
Nilai Angka      : 78.0
Nilai Huruf      : B
Keterangan       : Baik

```

Latihan 1.3 — Kalkulator BMI (Body Mass Index)

Buatlah program yang menghitung BMI dengan rumus $BMI = \text{berat} / (\text{tinggi}^2)$. Kategori: Kurus (<18.5), Normal (18.5-24.9), Gemuk (25-29.9), Obesitas (>=30).

 **Nama File:** latihan1_3_bmi.py

Kode Program:

```

1  print("===== Kalkulator BMI =====")
2  berat = float(input("Masukkan berat badan (kg) : "))
3  tinggi = float(input("Masukkan tinggi badan (m) : "))
4
5  bmi = berat / (tinggi ** 2)
6
7  if bmi < 18.5:
8      kategori = "Kurus"
9  elif bmi < 25:
10     kategori = "Normal"
11 elif bmi < 30:
12     kategori = "Gemuk"
13 else:
14     kategori = "Obesitas"
15
16 print("-" * 30)
17 print(f"Nilai BMI Anda : {bmi:.2f}")
18 print(f"Kategori      : {kategori}")

```

Hasil Eksekusi Program:

```

===== Kalkulator BMI =====
Masukkan berat badan (kg) : 65
Masukkan tinggi badan (m) : 1.70
-----
Nilai BMI Anda : 22.49
Kategori      : Normal

```

Latihan 1.4 — Tiket Bioskop dengan Diskon

Buatlah program harga tiket bioskop. Harga normal Rp. 50.000. Diskon 50% untuk umur <12 atau >60 tahun. Diskon 20% untuk pelajar (input Y/T).

 **Nama File:** latihan1_4_tiket.py

Kode Program:

```

1 print("==== Locket Tiket Bioskop =====")
2 harga_normal = 50000
3 umur = int(input("Masukkan umur      : "))
4 pelajar = input("Apakah pelajar (Y/T) : ").upper()
5
6 if umur < 12 or umur > 60:
7     diskon = 0.5
8     ket = "Diskon Anak/Lansia (50%)"
9 elif pelajar == "Y":
10     diskon = 0.2
11     ket = "Diskon Pelajar (20%)"
12 else:
13     diskon = 0
14     ket = "Harga Normal"
15
16 potongan = harga_normal * diskon
17 bayar = harga_normal - potongan
18
19 print("-" * 35)
20 print(f"Harga Normal : Rp {harga_normal:,}")
21 print(f"Keterangan   : {ket}")
22 print(f"Potongan      : Rp {int(potongan):,}")
23 print(f"Total Bayar   : Rp {int(bayar):,}")

```

Hasil Eksekusi Program:

```

==== Locket Tiket Bioskop =====
Masukkan umur      : 17
Apakah pelajar (Y/T) : Y
-----
Harga Normal : Rp 50,000
Keterangan   : Diskon Pelajar (20%)
Potongan      : Rp 10,000
Total Bayar   : Rp 40,000

```

Latihan 1.5 — Penentu Jenis Segitiga

Buatlah program yang menerima input tiga sisi segitiga kemudian menentukan jenisnya: sama sisi, sama kaki, atau sembarang. Program juga memvalidasi apakah ketiga sisi membentuk segitiga yang valid.

 **Nama File:** latihan1_5_segitiga.py

Kode Program:

```

1 print("==== Penentu Jenis Segitiga ====")
2 a = float(input("Masukkan sisi a : "))
3 b = float(input("Masukkan sisi b : "))
4 c = float(input("Masukkan sisi c : "))
5
6 # Validasi segitiga
7 if (a + b) <= c or (a + c) <= b or (b + c) <= a:
8     print("Ketiga sisi TIDAK membentuk segitiga!")
9 else:
10     if a == b == c:
11         jenis = "Segitiga Sama Sisi"
12     elif a == b or a == c or b == c:
13         jenis = "Segitiga Sama Kaki"
14     else:
15         jenis = "Segitiga Sembarang"
16
17 print("-" * 30)
18 print(f"Jenis Segitiga : {jenis}")

```

Hasil Eksekusi Program:

```

==== Penentu Jenis Segitiga ====
Masukkan sisi a : 5
Masukkan sisi b : 5
Masukkan sisi c : 8
-----
Jenis Segitiga : Segitiga Sama Kaki

```

 **Catatan:** Perhatikan penggunaan indentasi di Python — berbeda dengan bahasa lain yang memakai kurung kurawal, Python menggunakan indentasi (4 spasi atau 1 tab) untuk menandai blok kode.

BAB 2 PERULANGAN (LOOPING)

Tujuan Praktek:

- ◆ Memahami konsep perulangan (looping) dalam Python
- ◆ Mampu menggunakan perulangan for dan while dengan tepat
- ◆ Memahami penggunaan break dan continue dalam perulangan
- ◆ Mampu menyelesaikan masalah yang memerlukan proses berulang

2.1 Dasar Teori

Perulangan (looping) merupakan struktur kontrol yang digunakan untuk menjalankan satu atau sekumpulan perintah secara berulang-ulang hingga suatu kondisi terpenuhi. Python menyediakan dua bentuk utama perulangan: for dan while.

Perulangan for:

```
1 for variabel in iterable:
2     # blok kode yang akan diulang
```

Perulangan while:

```
1 while kondisi:
2     # blok kode yang akan diulang selama kondisi bernilai True
```

Fungsi range(start, stop, step) sering digunakan bersama for untuk menghasilkan urutan bilangan. Sementara itu, kata kunci break digunakan untuk menghentikan perulangan sebelum waktunya, dan continue untuk melewati iterasi saat ini dan lanjut ke iterasi berikutnya.

2.2 Latihan Praktikum

Latihan 2.1 — Tabel Perkalian

Buatlah program yang menampilkan tabel perkalian dari suatu angka yang diinput user, mulai dari 1 sampai 10.

 **Nama File:** latihan2_1_perkalian.py

Kode Program:

```
1 print("==== Tabel Perkalian =====")
2 n = int(input("Masukkan angka: "))
3 print("-" * 25)
4
5 for i in range(1, 11):
6     hasil = n * i
7     print(f"{n} x {i:2d} = {hasil}")
```

Hasil Eksekusi Program:

```

===== Tabel Perkalian =====
Masukkan angka: 7
-----
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

```

Latihan 2.2 — Menghitung Faktorial

Buatlah program yang menghitung faktorial dari bilangan bulat positif yang diinput. Faktorial $n! = 1 \times 2 \times 3 \times \dots \times n$.

 **Nama File:** `latihan2_2_faktorial.py`

Kode Program:

```

1 print("===== Menghitung Faktorial =====")
2 n = int(input("Masukkan bilangan bulat positif: "))
3
4 if n < 0:
5     print("Faktorial hanya untuk bilangan non-negatif!")
6 else:
7     hasil = 1
8     proses = ""
9     for i in range(1, n + 1):
10        hasil *= i
11        proses += f"{i}"
12        if i < n:
13            proses += " x "
14
15    if n == 0:
16        print(f"0! = 1")
17    else:
18        print(f"{n}! = {proses} = {hasil}")

```

Hasil Eksekusi Program:

```

===== Menghitung Faktorial =====
Masukkan bilangan bulat positif: 6
6! = 1 x 2 x 3 x 4 x 5 x 6 = 720

```

Latihan 2.3 — Deret Fibonacci

Buatlah program yang menampilkan deret Fibonacci sebanyak N suku. Deret Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, ...

 **Nama File:** latihan2_3_fibonacci.py

Kode Program:

```
1 print("==== Deret Fibonacci =====")
2 n = int(input("Masukkan jumlah suku: "))
3
4 a, b = 0, 1
5 print(f"Deret Fibonacci {n} suku pertama:")
6 for i in range(n):
7     print(a, end=" ")
8     a, b = b, a + b
9 print()
```

Hasil Eksekusi Program:

```
==== Deret Fibonacci =====
Masukkan jumlah suku: 10
Deret Fibonacci 10 suku pertama:
0 1 1 2 3 5 8 13 21 34
```

Latihan 2.4 — Menebak Angka (while + break)

Buatlah game sederhana: komputer memikirkan angka 1-20, user menebak. Program memberi petunjuk 'Terlalu besar' atau 'Terlalu kecil' sampai tebakan benar.

 **Nama File:** latihan2_4_tebak_angka.py

Kode Program:

```
1 import random
2
3 print("==== Game Tebak Angka =====")
4 print("Saya memikirkan angka 1 sampai 20")
5
6 angka_rahasia = random.randint(1, 20)
7 percobaan = 0
8
9 while True:
10     tebakan = int(input("Tebakan Anda: "))
11     percobaan += 1
12
13     if tebakan == angka_rahasia:
14         print(f"Benar! Anda menebak dalam {percobaan} percobaan.")
15         break
16     elif tebakan < angka_rahasia:
```

```
17     print("Terlalu kecil, coba lagi!")
18     else:
19         print("Terlalu besar, coba lagi!")
```

Hasil Eksekusi Program:

```
===== Game Tebak Angka =====
Saya memikirkan angka 1 sampai 20
Tebakan Anda: 10
Terlalu kecil, coba lagi!
Tebakan Anda: 15
Terlalu besar, coba lagi!
Tebakan Anda: 13
Terlalu besar, coba lagi!
Tebakan Anda: 12
Benar! Anda menebak dalam 4 percobaan.
```

Latihan 2.5 — Pola Piramida Angka

Buatlah program yang menampilkan pola piramida angka menggunakan perulangan bersarang (nested loop).

 **Nama File:** latihan2_5_piramida.py

Kode Program:

```
1  print("===== Pola Piramida Angka =====")
2  tinggi = int(input("Masukkan tinggi piramida: "))
3  print()
4
5  for i in range(1, tinggi + 1):
6      # Cetak spasi
7      for s in range(tinggi - i):
8          print(" ", end="")
9      # Cetak angka naik
10     for j in range(1, i + 1):
11         print(j, end="")
12     # Cetak angka turun
13     for j in range(i - 1, 0, -1):
14         print(j, end="")
15     print()
```

Hasil Eksekusi Program:

```
==== Pola Piramida Angka ====  
Masukkan tinggi piramida: 5  
  
    1  
   121  
  12321  
 1234321  
123454321
```

 **Catatan:** Perulangan while sangat cocok ketika jumlah iterasi tidak diketahui di awal, sedangkan for lebih cocok ketika jumlah iterasi sudah pasti atau kita ingin melakukan iterasi pada sebuah sequence/iterable.

BAB 3 STRUKTUR DATA DASAR

Tujuan Praktek:

- ◆ Memahami struktur data dasar di Python: List, Tuple, Dictionary, dan Set
- ◆ Mampu membuat, mengakses, dan memodifikasi data pada setiap struktur
- ◆ Mampu memilih struktur data yang tepat untuk menyelesaikan masalah
- ◆ Memahami operasi-operasi umum seperti menambah, menghapus, dan mengiterasi data

3.1 Dasar Teori

Python menyediakan beberapa struktur data built-in yang sangat powerful untuk mengelompokkan data. Masing-masing memiliki karakteristik dan penggunaan yang berbeda.

List

Struktur data teratur (ordered) dan dapat diubah (mutable). List ditulis dengan kurung siku []. Contoh: buah = ['apel', 'jeruk', 'mangga'].

Tuple

Mirip list tetapi tidak dapat diubah (immutable). Tuple ditulis dengan kurung bulat (). Contoh: koordinat = (10, 20).

Dictionary

Struktur data berupa pasangan key-value. Dictionary ditulis dengan kurung kurawal {}. Contoh: mhs = {'nama': 'Budi', 'nim': '2024001'}.

Set

Kumpulan elemen unik yang tidak teratur. Set ditulis dengan kurung kurawal {} tanpa pasangan key-value. Contoh: angka = {1, 2, 3, 4}.

3.2 Latihan Praktikum

Latihan 3.1 — Manajemen Daftar Belanja (List)

Buatlah program untuk mengelola daftar belanja. User dapat menambah, melihat, dan menghapus item dari daftar.

 **Nama File:** latihan3_1_list_belanja.py

Kode Program:

```
1 daftar_belanja = []
2
3 while True:
4     print("\n===== MENU DAFTAR BELANJA =====")
5     print("1. Tambah Item")
```

```

6     print("2. Lihat Daftar")
7     print("3. Hapus Item")
8     print("4. Keluar")
9     pilih = input("Pilihan: ")
10
11     if pilih == "1":
12         item = input("Nama item: ")
13         daftar_belanja.append(item)
14         print(f"'{item}' ditambahkan.")
15     elif pilih == "2":
16         if len(daftar_belanja) == 0:
17             print("Daftar masih kosong.")
18         else:
19             print("Isi daftar:")
20             for i, item in enumerate(daftar_belanja, 1):
21                 print(f" {i}. {item}")
22     elif pilih == "3":
23         item = input("Item yang ingin dihapus: ")
24         if item in daftar_belanja:
25             daftar_belanja.remove(item)
26             print(f"'{item}' dihapus.")
27         else:
28             print("Item tidak ditemukan.")
29     elif pilih == "4":
30         print("Terima kasih!")
31         break

```

Hasil Eksekusi Program:

```

===== MENU DAFTAR BELANJA =====
1. Tambah Item
2. Lihat Daftar
3. Hapus Item
4. Keluar
Pilihan: 1
Nama item: Beras
'Beras' ditambahkan.

===== MENU DAFTAR BELANJA =====
Pilihan: 1
Nama item: Minyak
'Minyak' ditambahkan.

===== MENU DAFTAR BELANJA =====
Pilihan: 2
Isi daftar:
1. Beras
2. Minyak

```

Latihan 3.2 — Statistika Nilai Mahasiswa (List)

Buatlah program yang menerima N nilai mahasiswa, lalu menampilkan nilai tertinggi, terendah, rata-rata, dan jumlah total.

 **Nama File:** latihan3_2_statistik.py

Kode Program:

```
1 print("==== Statistika Nilai Mahasiswa =====")
2 n = int(input("Jumlah data      : "))
3 nilai = []
4
5 for i in range(n):
6     x = float(input(f"Nilai ke-{i+1}      : "))
7     nilai.append(x)
8
9 total    = sum(nilai)
10 rata    = total / n
11 tertinggi = max(nilai)
12 terendah = min(nilai)
13
14 print("-" * 35)
15 print(f"Jumlah Nilai      : {total}")
16 print(f"Rata-rata          : {rata:.2f}")
17 print(f"Nilai Tertinggi     : {tertinggi}")
18 print(f"Nilai Terendah      : {terendah}")
```

Hasil Eksekusi Program:

```
==== Statistika Nilai Mahasiswa =====
Jumlah data      : 5
Nilai ke-1      : 80
Nilai ke-2      : 75
Nilai ke-3      : 90
Nilai ke-4      : 65
Nilai ke-5      : 85
-----
Jumlah Nilai      : 395.0
Rata-rata        : 79.00
Nilai Tertinggi   : 90.0
Nilai Terendah    : 65.0
```

Latihan 3.3 — Data Mahasiswa (Dictionary)

Buatlah program yang menyimpan data mahasiswa menggunakan dictionary, kemudian menampilkannya dalam format rapi.

 **Nama File:** latihan3_3_dictionary.py

Kode Program:

```

1 print("==== Input Data Mahasiswa =====")
2
3 mahasiswa = {
4     "nim"      : input("NIM      : "),
5     "nama"     : input("Nama     : "),
6     "jurusan"  : input("Jurusan  : "),
7     "ipk"      : float(input("IPK      : "))
8 }
9
10 print("\n===== DATA MAHASISWA =====")
11 for key, value in mahasiswa.items():
12     print(f"{key.capitalize():10s}: {value}")
13
14 if mahasiswa["ipk"] >= 3.5:
15     predikat = "Cumlaude"
16 elif mahasiswa["ipk"] >= 3.0:
17     predikat = "Sangat Memuaskan"
18 elif mahasiswa["ipk"] >= 2.5:
19     predikat = "Memuaskan"
20 else:
21     predikat = "Cukup"
22
23 print(f"{'Predikat':10s}: {predikat}")

```

Hasil Eksekusi Program:

```

==== Input Data Mahasiswa =====
NIM      : 2024010001
Nama     : Siti Nurhaliza
Jurusan  : Teknik Informatika
IPK      : 3.75

===== DATA MAHASISWA =====
Nim      : 2024010001
Nama     : Siti Nurhaliza
Jurusan  : Teknik Informatika
Ipk      : 3.75
Predikat : Cumlaude

```

Latihan 3.4 — Menghitung Frekuensi Huruf (Dictionary)

Buatlah program yang menghitung berapa kali setiap huruf muncul dalam sebuah kalimat menggunakan dictionary.

 **Nama File:** latihan3_4_frekuensi.py

Kode Program:

```
1 print("==== Frekuensi Huruf ====")
2 kalimat = input("Masukkan kalimat: ").lower()
3
4 frekuensi = {}
5 for huruf in kalimat:
6     if huruf.isalpha():
7         if huruf in frekuensi:
8             frekuensi[huruf] += 1
9         else:
10            frekuensi[huruf] = 1
11
12 # Urutkan berdasarkan abjad
13 print("\nHasil:")
14 for huruf in sorted(frekuensi.keys()):
15     print(f" '{huruf}' muncul {frekuensi[huruf]} kali")
```

Hasil Eksekusi Program:

```
==== Frekuensi Huruf ====
Masukkan kalimat: Python itu mudah

Hasil:
'a' muncul 1 kali
'd' muncul 1 kali
'h' muncul 2 kali
'i' muncul 1 kali
'm' muncul 1 kali
'n' muncul 1 kali
'o' muncul 1 kali
'p' muncul 1 kali
't' muncul 2 kali
'u' muncul 2 kali
'y' muncul 1 kali
```

Latihan 3.5 — Operasi Himpunan (Set)

Buatlah program yang menerima dua himpunan bilangan, kemudian menampilkan hasil operasi gabungan (union), irisan (intersection), dan selisih (difference).

 **Nama File:** latihan3_5_set.py

Kode Program:

```

1 print("==== Operasi Himpunan =====")
2
3 data_a = input("Anggota Himpunan A (pisah koma): ")
4 data_b = input("Anggota Himpunan B (pisah koma): ")
5
6 # Konversi ke set of integer
7 A = set(int(x.strip()) for x in data_a.split(","))
8 B = set(int(x.strip()) for x in data_b.split(","))
9
10 print("-" * 30)
11 print(f"A          = {A}")
12 print(f"B          = {B}")
13 print(f"A U B       = {A | B}")
14 print(f"A ∩ B        = {A & B}")
15 print(f"A - B        = {A - B}")
16 print(f"B - A        = {B - A}")
17 print(f"Simetris      = {A ^ B}")

```

Hasil Eksekusi Program:

```

==== Operasi Himpunan =====
Anggota Himpunan A (pisah koma): 1,2,3,4,5
Anggota Himpunan B (pisah koma): 4,5,6,7,8
-----
A          = {1, 2, 3, 4, 5}
B          = {4, 5, 6, 7, 8}
A U B       = {1, 2, 3, 4, 5, 6, 7, 8}
A ∩ B        = {4, 5}
A - B        = {1, 2, 3}
B - A        = {6, 7, 8}
Simetris     = {1, 2, 3, 6, 7, 8}

```

 **Catatan:** Pemilihan struktur data sangat penting. Gunakan List untuk data terurut yang dapat berubah, Tuple untuk data yang tetap, Dictionary untuk data berpasangan key-value, dan Set untuk data unik tanpa urutan.

BAB 4 FUNGSI (FUNCTION)

Tujuan Praktek:

- ◆ Memahami konsep fungsi dan manfaatnya dalam pemrograman
- ◆ Mampu membuat fungsi dengan parameter dan nilai kembali (return)
- ◆ Memahami perbedaan parameter dan argumen
- ◆ Memahami konsep scope variabel (lokal dan global)
- ◆ Mampu menggunakan default parameter dan keyword argument

4.1 Dasar Teori

Fungsi (function) adalah blok kode yang dapat dipanggil berkali-kali untuk melakukan tugas tertentu. Fungsi membantu membuat kode lebih terstruktur, mudah dibaca, dan dapat digunakan ulang (reusable). Di Python, fungsi didefinisikan dengan kata kunci `def`.

Bentuk umum fungsi:

```
1 def nama_fungsi(parameter1, parameter2, ...):
2     # blok kode fungsi
3     return nilai_kembali # opsional
```

Fungsi dapat memiliki parameter (nilai masukan) dan return value (nilai keluaran). Jika fungsi tidak memiliki return, secara otomatis akan mengembalikan `None`. Parameter dapat diberi nilai default sehingga tidak wajib diisi saat pemanggilan.

4.2 Latihan Praktikum

Latihan 4.1 — Fungsi Konversi Suhu

Buatlah fungsi untuk mengkonversi suhu antara Celcius, Fahrenheit, dan Reamur menggunakan tiga fungsi terpisah.

 **Nama File:** `latihan4_1_suhu.py`

Kode Program:

```
1 def celcius_to_fahrenheit(c):
2     return (c * 9/5) + 32
3
4 def celcius_to_reamur(c):
5     return c * 4/5
6
7 def celcius_to_kelvin(c):
8     return c + 273.15
9
10 print("==== Konversi Suhu =====")
```

```

11 c = float(input("Masukkan suhu dalam Celcius: "))
12
13 print("-" * 30)
14 print(f"{c}°C = {celcius_to_fahrenheit(c):.2f}°F")
15 print(f"{c}°C = {celcius_to_reamur(c):.2f}°R")
16 print(f"{c}°C = {celcius_to_kelvin(c):.2f} K")

```

Hasil Eksekusi Program:

```

===== Konversi Suhu =====
Masukkan suhu dalam Celcius: 30
-----
30.0°C = 86.00°F
30.0°C = 24.00°R
30.0°C = 303.15 K

```

Latihan 4.2 — Fungsi Cek Bilangan Prima

Buatlah fungsi yang mengembalikan True jika sebuah bilangan adalah prima, dan False jika bukan. Gunakan fungsi tersebut untuk mencetak bilangan prima dari 1 sampai N.

 **Nama File:** latihan4_2_prima.py

Kode Program:

```

1 def is_prima(n):
2     if n < 2:
3         return False
4     for i in range(2, int(n ** 0.5) + 1):
5         if n % i == 0:
6             return False
7     return True
8
9 print("===== Daftar Bilangan Prima =====")
10 batas = int(input("Masukkan batas atas: "))
11
12 print(f"\nBilangan prima dari 1 sampai {batas}:")
13 hasil = []
14 for i in range(1, batas + 1):
15     if is_prima(i):
16         hasil.append(i)
17
18 print(hasil)
19 print(f"Total: {len(hasil)} bilangan prima")

```

Hasil Eksekusi Program:

```

===== Daftar Bilangan Prima =====
Masukkan batas atas: 30

Bilangan prima dari 1 sampai 30:
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
Total: 10 bilangan prima

```

Latihan 4.3 — Fungsi dengan Default Parameter

Buatlah fungsi penghitung harga setelah diskon dengan parameter default. Diskon default 10%, pajak default 11%.

 **Nama File:** latihan4_3_diskon.py

Kode Program:

```

1 def hitung_harga(harga_awal, diskon=10, pajak=11):
2     potongan = harga_awal * diskon / 100
3     setelah_diskon = harga_awal - potongan
4     nilai_pajak = setelah_diskon * pajak / 100
5     total = setelah_diskon + nilai_pajak
6     return total, potongan, nilai_pajak
7
8 print("===== Kalkulator Harga =====")
9 harga = float(input("Harga barang   : "))
10 d     = float(input("Diskon (%)    : "))
11
12 total, pot, pjk = hitung_harga(harga, d)
13
14 print("-" * 35)
15 print(f"Harga Awal       : Rp {harga:,.0f}")
16 print(f"Potongan         : Rp {pot:,.0f}")
17 print(f"Pajak (11%)       : Rp {pjk:,.0f}")
18 print(f"Total Bayar      : Rp {total:,.0f}")

```

Hasil Eksekusi Program:

```

===== Kalkulator Harga =====
Harga barang   : 200000
Diskon (%)     : 15
-----
Harga Awal     : Rp 200,000
Potongan       : Rp 30,000
Pajak (11%)    : Rp 18,700
Total Bayar    : Rp 188,700

```

Latihan 4.4 — Fungsi Rekursif Faktorial

Buatlah fungsi rekursif untuk menghitung faktorial. Fungsi rekursif adalah fungsi yang memanggil dirinya sendiri.

 **Nama File:** latihan4_4_rekursif.py

Kode Program:

```

1 def faktorial(n):
2     # Base case
3     if n <= 1:
4         return 1
5     # Recursive case
6     return n * faktorial(n - 1)
7
8 def pangkat(basis, eksponen):
9     if eksponen == 0:
10        return 1
11    return basis * pangkat(basis, eksponen - 1)
12
13 print("==== Fungsi Rekursif =====")
14 n = int(input("Hitung faktorial dari: "))
15 b = int(input("Basis pangkat      : "))
16 e = int(input("Eksponen pangkat   : "))
17
18 print("-" * 30)
19 print(f"{n}! = {faktorial(n)}")
20 print(f"{b}^{e} = {pangkat(b, e)}")

```

Hasil Eksekusi Program:

```

==== Fungsi Rekursif =====
Hitung faktorial dari: 7
Basis pangkat      : 2
Eksponen pangkat   : 10
-----
7! = 5040
2^10 = 1024

```

Latihan 4.5 — Kalkulator Fungsional

Buatlah kalkulator yang memanfaatkan beberapa fungsi untuk operasi matematika. Setiap operasi ditangani oleh fungsi terpisah.

 **Nama File:** latihan4_5_kalkulator.py

Kode Program:

```

1 def tambah(a, b):    return a + b
2 def kurang(a, b):    return a - b

```

```

3 def kali(a, b):      return a * b
4 def bagi(a, b):
5     if b == 0:
6         return None
7     return a / b
8
9 def tampilkan_menu():
10    print("\n===== KALKULATOR =====")
11    print("1. Tambah")
12    print("2. Kurang")
13    print("3. Kali")
14    print("4. Bagi")
15    print("5. Keluar")
16
17 while True:
18     tampilkan_menu()
19     pilih = input("Pilihan: ")
20     if pilih == "5":
21         print("Terima kasih!")
22         break
23     if pilih not in "1234":
24         print("Pilihan tidak valid!")
25         continue
26
27     a = float(input("Angka pertama : "))
28     b = float(input("Angka kedua   : "))
29
30     if pilih == "1":  hasil = tambah(a, b); op = "+"
31     elif pilih == "2": hasil = kurang(a, b); op = "-"
32     elif pilih == "3": hasil = kali(a, b);   op = "x"
33     else:             hasil = bagi(a, b);   op = "/"
34
35     if hasil is None:
36         print("Error: Pembagian dengan nol!")
37     else:
38         print(f"Hasil: {a} {op} {b} = {hasil}")

```

Hasil Eksekusi Program:

```

===== KALKULATOR =====
1. Tambah
2. Kurang
3. Kali
4. Bagi
5. Keluar
Pilihan: 3
Angka pertama : 12
Angka kedua   : 8
Hasil: 12.0 x 8.0 = 96.0

```

```
===== KALKULATOR =====  
Pilihan: 5  
Terima kasih!
```

 **Catatan:** Prinsip DRY (Don't Repeat Yourself) mendorong kita memakai fungsi untuk menghindari pengulangan kode. Selain itu, fungsi yang baik melakukan satu hal saja dengan jelas (Single Responsibility Principle).

BAB 5 ALGORITMA DAN LOGIKA PEMROGRAMAN

Tujuan Praktek:

- ◆ Memahami konsep algoritma dasar seperti pencarian dan pengurutan
- ◆ Mampu mengimplementasikan algoritma linear search dan bubble sort
- ◆ Memahami cara berpikir algoritmik (algorithmic thinking)
- ◆ Mampu memecah masalah kompleks menjadi langkah-langkah logis

5.1 Dasar Teori

Algoritma adalah urutan langkah-langkah logis untuk menyelesaikan suatu masalah. Algoritma yang baik harus memenuhi beberapa kriteria: memiliki input dan output yang jelas, efektif (dapat dilaksanakan), terbatas (finite), dan tidak ambigu.

Pada bab ini akan dibahas beberapa algoritma klasik yang sering digunakan, yaitu algoritma pencarian (searching) dan pengurutan (sorting). Linear Search mencari elemen satu per satu hingga ditemukan, sedangkan Bubble Sort mengurutkan data dengan membandingkan pasangan elemen yang berdekatan.

5.2 Latihan Praktikum

Latihan 5.1 — Linear Search

Implementasikan algoritma Linear Search untuk mencari posisi sebuah elemen dalam sebuah list.

 **Nama File:** latihan5_1_linear_search.py

Kode Program:

```

1 def linear_search(data, target):
2     for i in range(len(data)):
3         if data[i] == target:
4             return i
5     return -1
6
7 print("==== Algoritma Linear Search =====")
8 data = [15, 27, 8, 42, 33, 19, 5, 66, 21]
9 print(f>Data      : {data}")
10
11 target = int(input("Cari angka: "))
12 posisi = linear_search(data, target)
13
14 if posisi != -1:
15     print(f"Angka {target} ditemukan pada indeks ke-{posisi}")
16 else:
17     print(f"Angka {target} TIDAK ditemukan dalam data")

```

Hasil Eksekusi Program:

```

===== Algoritma Linear Search =====
Data      : [15, 27, 8, 42, 33, 19, 5, 66, 21]
Cari angka: 33
Angka 33 ditemukan pada indeks ke-4

```

Latihan 5.2 — Bubble Sort

Implementasikan algoritma Bubble Sort untuk mengurutkan data secara ascending (dari kecil ke besar). Program juga menampilkan tahapan pengurutan.

 **Nama File:** latihan5_2_bubble_sort.py

Kode Program:

```

1 def bubble_sort(data):
2     n = len(data)
3     langkah = 1
4     for i in range(n):
5         swap = False
6         for j in range(0, n - i - 1):
7             if data[j] > data[j + 1]:
8                 data[j], data[j + 1] = data[j + 1], data[j]
9                 swap = True
10        print(f"Iterasi {langkah}: {data}")
11        langkah += 1
12        if not swap:
13            break
14    return data
15
16 print("===== Algoritma Bubble Sort =====")
17 data = [64, 34, 25, 12, 22, 11, 90]
18 print(f>Data awal : {data}\n")
19
20 hasil = bubble_sort(data.copy())
21 print(f"\nData akhir: {hasil}")

```

Hasil Eksekusi Program:

```

===== Algoritma Bubble Sort =====
Data awal : [64, 34, 25, 12, 22, 11, 90]

Iterasi 1: [34, 25, 12, 22, 11, 64, 90]
Iterasi 2: [25, 12, 22, 11, 34, 64, 90]
Iterasi 3: [12, 22, 11, 25, 34, 64, 90]
Iterasi 4: [12, 11, 22, 25, 34, 64, 90]
Iterasi 5: [11, 12, 22, 25, 34, 64, 90]
Iterasi 6: [11, 12, 22, 25, 34, 64, 90]

Data akhir: [11, 12, 22, 25, 34, 64, 90]

```

Latihan 5.3 — FPB dan KPK (Algoritma Euclidean)

Buatlah program yang mencari FPB (Faktor Persekutuan Terbesar) menggunakan algoritma Euclidean, lalu gunakan untuk menghitung KPK.

 **Nama File:** latihan5_3_fpb_kpk.py

Kode Program:

```

1 def fpb(a, b):
2     # Algoritma Euclidean
3     while b != 0:
4         a, b = b, a % b
5     return a
6
7 def kpk(a, b):
8     return (a * b) // fpb(a, b)
9
10 print("==== Kalkulator FPB dan KPK ====")
11 x = int(input("Bilangan pertama: "))
12 y = int(input("Bilangan kedua : "))
13
14 print("-" * 35)
15 print(f"FPB({x}, {y}) = {fpb(x, y)}")
16 print(f"KPK({x}, {y}) = {kpk(x, y)}")

```

Hasil Eksekusi Program:

```

==== Kalkulator FPB dan KPK ====
Bilangan pertama: 24
Bilangan kedua : 36
-----
FPB(24, 36) = 12
KPK(24, 36) = 72

```

Latihan 5.4 — Palindrom

Buatlah program yang memeriksa apakah suatu kata atau kalimat adalah palindrom (dibaca sama dari depan maupun belakang).

 **Nama File:** latihan5_4_palindrom.py

Kode Program:

```

1 def is_palindrom(teks):
2     # Bersihkan teks: hapus spasi dan jadikan lowercase
3     bersih = teks.replace(" ", "").lower()
4     # Bandingkan dengan kebalikannya
5     return bersih == bersih[::-1]
6
7 print("==== Pengecek Palindrom ====")
8 teks = input("Masukkan kata/kalimat: ")
9

```

```

10 if is_palindrom(teks):
11     print(f"'{teks}' adalah PALINDROM")
12 else:
13     print(f"'{teks}' BUKAN palindrom")
14
15 # Demo beberapa contoh
16 print("\n--- Demo ---")
17 contoh = ["kasur ini rusak", "katak", "python", "malam", "saya"]
18 for c in contoh:
19     status = "YA" if is_palindrom(c) else "BUKAN"
20     print(f" '{c}' -> {status} palindrom")

```

Hasil Eksekusi Program:

```

===== Pengecek Palindrom =====
Masukkan kata/kalimat: kasur ini rusak
'kasur ini rusak' adalah PALINDROM

--- Demo ---
'kasur ini rusak' -> YA palindrom
'katak' -> YA palindrom
'python' -> BUKAN palindrom
'malam' -> YA palindrom
'saya' -> BUKAN palindrom

```

Latihan 5.5 — Konversi Bilangan Desimal ke Biner

Buatlah program yang mengkonversi bilangan desimal menjadi biner menggunakan metode pembagian berturut-turut.

 **Nama File:** latihan5_5_biner.py

Kode Program:

```

1 def desimal_to_biner(n):
2     if n == 0:
3         return "0"
4     biner = ""
5     proses = []
6     while n > 0:
7         sisa = n % 2
8         proses.append(f"{n} / 2 = {n//2} sisa {sisa}")
9         biner = str(sisa) + biner
10        n = n // 2
11    return biner, proses
12
13 print("===== Konversi Desimal ke Biner =====")
14 n = int(input("Masukkan bilangan desimal: "))
15
16 biner, proses = desimal_to_biner(n)
17
18 print("\nProses konversi:")

```

```
19 for p in proses:
20     print(f" {p}")
21
22 print(f"\nHasil: {biner} (biner)")
23 print(f"Verifikasi: {int(biner, 2)} (desimal)")
```

Hasil Eksekusi Program:

```
===== Konversi Desimal ke Biner =====
Masukkan bilangan desimal: 25

Proses konversi:
  25 / 2 = 12 sisa 1
  12 / 2 = 6 sisa 0
  6 / 2 = 3 sisa 0
  3 / 2 = 1 sisa 1
  1 / 2 = 0 sisa 1

Hasil: 11001 (biner)
Verifikasi: 25 (desimal)
```

 **Catatan:** Sebelum menulis kode, biasakan merancang algoritma terlebih dahulu (dalam bentuk flowchart atau pseudocode). Hal ini membantu kita berpikir sistematis dan mengurangi kesalahan logika saat implementasi.

BAB 6 PENGOLAHAN FILE SEDERHANA

Tujuan Praktek:

- ◆ Memahami cara membuka, menulis, dan membaca file teks di Python
- ◆ Memahami mode pembukaan file (r, w, a, r+)
- ◆ Mampu menggunakan context manager (with statement)
- ◆ Mampu membuat aplikasi sederhana yang menyimpan data ke file

6.1 Dasar Teori

Pengolahan file adalah cara program berinteraksi dengan penyimpanan permanen. Berbeda dengan variabel yang hanya ada selama program berjalan, data dalam file tetap ada setelah program ditutup.

Fungsi `open()` memiliki beberapa mode:

Mode 'r' untuk membaca (read, default), 'w' untuk menulis dan menimpa file (write), 'a' untuk menambahkan di akhir file (append), 'r+' untuk baca dan tulis.

Rekomendasi penggunaan `with statement`:

```
1 with open("data.txt", "w") as f:
2     f.write("Hello, World!")
3 # File otomatis tertutup setelah blok selesai
```

Menggunakan `with` membuat file otomatis ditutup meskipun terjadi error, sehingga lebih aman daripada pemanggilan `open()` dan `close()` manual.

6.2 Latihan Praktikum

Latihan 6.1 — Menulis File Teks

Buatlah program yang menulis beberapa baris data ke dalam file teks. Data bisa berupa biodata atau catatan sederhana.

 **Nama File:** `latihan6_1_tulis_file.py`

Kode Program:

```
1 print("==== Menulis File Biodata =====")
2 nama    = input("Nama      : ")
3 umur    = input("Umur      : ")
4 alamat  = input("Alamat    : ")
5 hobi    = input("Hobi      : ")
6
7 with open("biodata.txt", "w") as f:
8     f.write("==== BIODATA =====\n")
```

```

9     f.write(f>Nama    : {nama}\n")
10    f.write(f"Umur    : {umur}\n")
11    f.write(f"Alamat  : {alamat}\n")
12    f.write(f"Hobi    : {hobi}\n")
13
14    print("\nFile 'biodata.txt' berhasil dibuat!")

```

Hasil Eksekusi Program:

```

===== Menulis File Biodata =====
Nama      : Rudi Santoso
Umur      : 20
Alamat    : Banjarmasin
Hobi      : Membaca

File 'biodata.txt' berhasil dibuat!

```

Latihan 6.2 — Membaca File Teks

Buatlah program yang membaca file teks yang telah dibuat sebelumnya dan menampilkan seluruh isinya.

 **Nama File:** latihan6_2_baca_file.py

Kode Program:

```

1    print("===== Membaca File Biodata =====\n")
2
3    try:
4        with open("biodata.txt", "r") as f:
5            isi = f.read()
6            print(isi)
7
8        # Baca per baris
9        print("--- Per Baris ---")
10       with open("biodata.txt", "r") as f:
11           for nomor, baris in enumerate(f, 1):
12               print(f"Baris {nomor}: {baris.rstrip()}")
13   except FileNotFoundError:
14       print("File tidak ditemukan! Jalankan latihan 6.1 terlebih dahulu.")

```

Hasil Eksekusi Program:

```

===== Membaca File Biodata =====

===== BIODATA =====
Nama      : Rudi Santoso

```

```

Umur   : 20
Alamat : Banjarmasin
Hobi   : Membaca

--- Per Baris ---
Baris 1: ===== BIODATA =====
Baris 2: Nama      : Rudi Santoso
Baris 3: Umur      : 20
Baris 4: Alamat    : Banjarmasin
Baris 5: Hobi      : Membaca

```

Latihan 6.3 — Append Data (Catatan Harian)

Buatlah program catatan harian yang menambahkan catatan baru ke file tanpa menghapus catatan lama. Gunakan mode 'a' (append).

 **Nama File:** latihan6_3_catatan.py

Kode Program:

```

1  from datetime import datetime
2
3  print("===== Buku Catatan Harian =====")
4
5  while True:
6      catatan = input("\nTulis catatan (ketik 'selesai' untuk berhenti): ")
7      if catatan.lower() == "selesai":
8          break
9
10     waktu = datetime.now().strftime("%Y-%m-%d %H:%M")
11
12     with open("catatan.txt", "a") as f:
13         f.write(f"[{waktu}] {catatan}\n")
14
15     print("✓ Tersimpan")
16
17     print("\n===== Isi catatan.txt =====")
18     try:
19         with open("catatan.txt", "r") as f:
20             print(f.read())
21     except FileNotFoundError:
22         print("(belum ada catatan)")

```

Hasil Eksekusi Program:

```

===== Buku Catatan Harian =====

Tulis catatan (ketik 'selesai' untuk berhenti): Belajar Python hari ini
✓ Tersimpan

Tulis catatan (ketik 'selesai' untuk berhenti): Praktikum Algoritma selesai

```

✓ Tersimpan

Tulis catatan (ketik 'selesai' untuk berhenti): selesai

```
===== Isi catatan.txt =====
[2026-04-23 10:15] Belajar Python hari ini
[2026-04-23 10:16] Praktikum Algoritma selesai
```

Latihan 6.4 — Menghitung Statistik File

Buatlah program yang membaca file teks lalu menghitung jumlah baris, jumlah kata, dan jumlah karakter.

 **Nama File:** latihan6_4_statistik_file.py

Kode Program:

```
1 print("===== Statistik File Teks =====")
2 nama_file = input("Nama file: ")
3
4 try:
5     with open(nama_file, "r") as f:
6         isi = f.read()
7
8         jumlah_baris    = isi.count("\n") + (0 if isi.endswith("\n") else 1)
9         jumlah_kata     = len(isi.split())
10        jumlah_karakter = len(isi)
11        jumlah_huruf    = sum(1 for c in isi if c.isalpha())
12
13    print("-" * 35)
14    print(f"File           : {nama_file}")
15    print(f"Jumlah baris    : {jumlah_baris}")
16    print(f"Jumlah kata     : {jumlah_kata}")
17    print(f"Jumlah karakter : {jumlah_karakter}")
18    print(f"Jumlah huruf    : {jumlah_huruf}")
19 except FileNotFoundError:
20    print(f"File '{nama_file}' tidak ditemukan!")
```

Hasil Eksekusi Program:

```
===== Statistik File Teks =====
Nama file: biodata.txt
-----
File           : biodata.txt
Jumlah baris    : 5
Jumlah kata     : 14
Jumlah karakter : 95
Jumlah huruf    : 68
```

Latihan 6.5 — Database Sederhana dengan CSV

Buatlah aplikasi sederhana yang menyimpan data mahasiswa dalam format CSV (comma-separated values). Program dapat menambah dan menampilkan semua data.

 **Nama File:** latihan6_5_csv.py

Kode Program:

```

1  import os
2
3  FILE = "mahasiswa.csv"
4
5  def tambah_data():
6      nim    = input("NIM      : ")
7      nama   = input("Nama     : ")
8      jurusan = input("Jurusan  : ")
9      ipk    = input("IPK      : ")
10
11     # Buat header jika file belum ada
12     header = not os.path.exists(FILE)
13     with open(FILE, "a") as f:
14         if header:
15             f.write("NIM>Nama>Jurusan>IPK\n")
16             f.write(f"{nim}>{nama}>{jurusan}>{ipk}\n")
17     print("✓ Data tersimpan")
18
19     def tampil_data():
20         if not os.path.exists(FILE):
21             print("Belum ada data.")
22             return
23         with open(FILE, "r") as f:
24             print("\n" + "-" * 55)
25             for i, baris in enumerate(f):
26                 data = baris.strip().split(">")
27                 if i == 0:
28                     print(f"{data[0]:12s} {data[1]:20s} {data[2]:12s} {data[3]:>5s}")
29                     print("-" * 55)
30                 else:
31                     print(f"{data[0]:12s} {data[1]:20s} {data[2]:12s} {data[3]:>5s}")
32
33     while True:
34         print("\n1. Tambah Data")
35         print("2. Lihat Data")
36         print("3. Keluar")
37         p = input("Pilihan: ")
38         if p == "1": tambah_data()
39         elif p == "2": tampil_data()
40         elif p == "3": break

```

Hasil Eksekusi Program:

```
1. Tambah Data
2. Lihat Data
3. Keluar
Pilihan: 1
NIM      : 2024001
Nama     : Andi Pratama
Jurusan  : TI
IPK      : 3.45
✓ Data tersimpan
```

```
Pilihan: 1
NIM      : 2024002
Nama     : Sari Dewi
Jurusan  : SI
IPK      : 3.78
✓ Data tersimpan
```

```
Pilihan: 2
```

```
-----
NIM      Nama           Jurusan   IPK
-----
2024001  Andi Pratama         TI        3.45
2024002  Sari Dewi            SI        3.78
```

 **Catatan:** Selalu gunakan 'with open(...) as f:' untuk menangani file. Konstruksi ini memastikan file otomatis ditutup meskipun terjadi kesalahan di tengah proses, sehingga mencegah kebocoran resource.

BAB 7 DEBUGGING DAN ERROR HANDLING

Tujuan Praktek:

- ◆ Memahami jenis-jenis error dalam Python: *SyntaxError, RuntimeError, LogicError*
- ◆ Mampu menggunakan blok *try-except-finally* untuk menangani *exception*
- ◆ Mampu membuat *exception handler* yang spesifik dan informatif
- ◆ Memahami teknik *debugging* dasar menggunakan *print statement*
- ◆ Memahami penggunaan *raise* untuk melempar *exception* kustom

7.1 Dasar Teori

Dalam pemrograman, error (kesalahan) adalah hal yang wajar terjadi. Terdapat tiga jenis error utama di Python:

1. Syntax Error

Kesalahan penulisan kode yang tidak sesuai aturan bahasa Python. Program tidak akan berjalan sama sekali sebelum syntax diperbaiki.

2. Runtime Error (Exception)

Error yang terjadi saat program berjalan. Contoh: *ZeroDivisionError, ValueError, FileNotFoundError, TypeError*.

3. Logic Error

Program berjalan tanpa error tapi memberikan hasil yang salah. Ini biasanya paling sulit ditemukan karena butuh pengecekan logika program.

Struktur *try-except*:

```

1 try:
2     # kode yang mungkin menimbulkan error
3 except JenisError as e:
4     # penanganan error
5 else:
6     # dijalankan jika tidak ada error
7 finally:
8     # selalu dijalankan

```

7.2 Latihan Praktikum

Latihan 7.1 — Validasi Input Numerik

Buatlah program yang meminta input angka dari user. Jika user memasukkan bukan angka, program tidak crash tapi meminta ulang.

 Nama File: `latihan7_1_validasi.py`

Kode Program:

```

1 print("==== Validasi Input =====")
2
3 while True:
4     try:
5         angka = float(input("Masukkan angka: "))
6         break
7     except ValueError:
8         print("✗ Input bukan angka, coba lagi!")
9
10 print(f"✓ Anda memasukkan: {angka}")
11 print(f"  Kuadratnya      : {angka ** 2}")

```

Hasil Eksekusi Program:

```

==== Validasi Input =====
Masukkan angka: abc
✗ Input bukan angka, coba lagi!
Masukkan angka: 12x
✗ Input bukan angka, coba lagi!
Masukkan angka: 7.5
✓ Anda memasukkan: 7.5
  Kuadratnya      : 56.25

```

Latihan 7.2 — Menangani Pembagian Nol

Buatlah program kalkulator pembagian yang dapat menangani pembagian dengan nol (ZeroDivisionError).

 **Nama File:** latihan7_2_bagi_nol.py

Kode Program:

```

1 print("==== Kalkulator Pembagian =====")
2
3 try:
4     a = float(input("Pembilang : "))
5     b = float(input("Penyebut  : "))
6     hasil = a / b
7 except ZeroDivisionError:
8     print("✗ Error: Tidak bisa dibagi dengan nol!")
9 except ValueError:
10    print("✗ Error: Input harus berupa angka!")
11 else:
12    print(f"✓ {a} / {b} = {hasil:.4f}")
13 finally:
14    print("(Proses selesai)")

```

Hasil Eksekusi Program:

```

===== Kalkulator Pembagian =====
Pembilang : 15
Penyebut  : 0
❌ Error: Tidak bisa dibagi dengan nol!
(Proses selesai)

```

Latihan 7.3 — Menangani File Tidak Ditemukan

Buatlah program yang mencoba membaca sebuah file. Jika file tidak ada, tampilkan pesan yang informatif tanpa crash.

 **Nama File:** latihan7_3_file_error.py

Kode Program:

```

1 print("===== Pembaca File =====")
2 nama_file = input("Nama file: ")
3
4 try:
5     with open(nama_file, "r") as f:
6         isi = f.read()
7         print("\n--- Isi File ---")
8         print(isi)
9 except FileNotFoundError:
10    print(f"❌ File '{nama_file}' tidak ditemukan.")
11    pilih = input("Buat file baru? (Y/T): ").upper()
12    if pilih == "Y":
13        konten = input("Tulis isi file: ")
14        with open(nama_file, "w") as f:
15            f.write(konten)
16        print(f"✓ File '{nama_file}' dibuat.")
17 except PermissionError:
18    print(f"❌ Tidak punya izin akses ke '{nama_file}'.")
19 except Exception as e:
20    print(f"❌ Error tak terduga: {type(e).__name__}: {e}")

```

Hasil Eksekusi Program:

```

===== Pembaca File =====
Nama file: data_hilang.txt
❌ File 'data_hilang.txt' tidak ditemukan.
Buat file baru? (Y/T): Y
Tulis isi file: Ini file baru
✓ File 'data_hilang.txt' dibuat.

```

Latihan 7.4 — Custom Exception dengan raise

Buatlah program validasi umur dengan aturan: umur harus 0-120. Gunakan raise untuk melempar exception jika tidak valid.

 **Nama File:** latihan7_4_raise.py

Kode Program:

```
1 def validasi_umur(umur):
2     if not isinstance(umur, (int, float)):
3         raise TypeError("Umur harus berupa angka")
4     if umur < 0:
5         raise ValueError("Umur tidak boleh negatif")
6     if umur > 120:
7         raise ValueError("Umur tidak realistis (>120)")
8     return True
9
10 print("==== Validasi Umur =====")
11
12 try:
13     umur = int(input("Umur Anda: "))
14     validasi_umur(umur)
15     print(f"✓ Umur {umur} valid")
16
17     if umur < 17:
18         print(" Kategori: Anak-anak/Remaja")
19     elif umur < 60:
20         print(" Kategori: Dewasa")
21     else:
22         print(" Kategori: Lansia")
23 except ValueError as e:
24     print(f"✗ Error: {e}")
25 except TypeError as e:
26     print(f"✗ Error: {e}")
```

Hasil Eksekusi Program:

```
==== Validasi Umur =====
Umur Anda: 150
✗ Error: Umur tidak realistis (>120)
```

Latihan 7.5 — Debugging dengan Print Statement

Program berikut memiliki logic error. Gunakan print statement untuk men-debug dan menemukan kesalahannya. Program seharusnya menghitung rata-rata.

 **Nama File:** latihan7_5_debug.py

Kode Program:

```

1 def rata_rata(data):
2     print(f"[DEBUG] Data masuk : {data}")
3     print(f"[DEBUG] Jumlah item: {len(data)}")
4
5     total = 0
6     for i, angka in enumerate(data):
7         total += angka
8         print(f"[DEBUG] Iterasi {i}: total = {total}")
9
10    if len(data) == 0:
11        print("[DEBUG] Data kosong, return 0")
12        return 0
13
14    hasil = total / len(data)
15    print(f"[DEBUG] Total akhir: {total}")
16    print(f"[DEBUG] Rata-rata : {hasil}")
17    return hasil
18
19 print("==== Program Rata-rata (with debug) ====")
20
21 data1 = [80, 75, 90, 85, 70]
22 print(f"\nData: {data1}")
23 r = rata_rata(data1)
24 print(f"\n==> Rata-rata = {r}")

```

Hasil Eksekusi Program:

```

==== Program Rata-rata (with debug) ====

Data: [80, 75, 90, 85, 70]
[DEBUG] Data masuk : [80, 75, 90, 85, 70]
[DEBUG] Jumlah item: 5
[DEBUG] Iterasi 0: total = 80
[DEBUG] Iterasi 1: total = 155
[DEBUG] Iterasi 2: total = 245
[DEBUG] Iterasi 3: total = 330
[DEBUG] Iterasi 4: total = 400
[DEBUG] Total akhir: 400
[DEBUG] Rata-rata : 80.0

==> Rata-rata = 80.0

```

 **Catatan:** Selalu tangkap exception yang spesifik (ValueError, TypeError) daripada menggunakan 'except:' kosong. Exception yang terlalu umum dapat menyembunyikan bug yang seharusnya muncul ke permukaan untuk diperbaiki.

BAB 8 MINI PROJECT (STUDI KASUS)

Tujuan Praktek:

- ◆ Mengintegrasikan seluruh materi yang sudah dipelajari dalam sebuah program lengkap
- ◆ Mampu menganalisis kasus nyata dan menerjemahkannya ke dalam kode Python
- ◆ Mampu membuat aplikasi yang memiliki fitur CRUD (Create, Read, Update, Delete)
- ◆ Mampu membuat fitur pencetakan laporan/struk dari data yang tersimpan

8.1 Deskripsi Studi Kasus

Aplikasi Kasir Toko Sederhana

Pada bagian ini, kita akan membuat sebuah aplikasi kasir sederhana untuk sebuah toko kelontong. Aplikasi ini menggabungkan seluruh konsep yang sudah dipelajari: percabangan, perulangan, struktur data, fungsi, pengolahan file, serta error handling. Aplikasi juga dilengkapi fitur cetak struk belanja yang disimpan dalam file teks.

Fitur yang tersedia:

1. Tambah barang ke daftar produk. 2. Lihat daftar produk. 3. Mulai transaksi pembelian (keranjang).
4. Hitung total dan proses pembayaran. 5. Cetak struk ke file teks dan layar. 6. Lihat riwayat transaksi.

8.2 Source Code Lengkap

 **Nama File:** `mini_project_kasir.py`

Kode Program:

```

1  import os
2  from datetime import datetime
3
4  # ===== FILE KONFIGURASI =====
5  FILE_PRODUK   = "produk.txt"
6  FILE_RIWAYAT  = "riwayat_transaksi.txt"
7  FOLDER_STRUK  = "struk"
8
9  # ===== UTILITAS =====
10 def pastikan_folder():
11     if not os.path.exists(FOLDER_STRUK):
12         os.makedirs(FOLDER_STRUK)
13
14 def format_rupiah(angka):
15     return f"Rp {int(angka):,}".replace(",", ".")
16
17 def input_angka(prompt, tipe=int):
18     while True:

```

```

19         try:
20             return tipe(input(prompt))
21         except ValueError:
22             print("✗ Input harus berupa angka!")
23
24     # ===== MANAJEMEN PRODUK =====
25     def muat_produk():
26         produk = {}
27         if not os.path.exists(FILE_PRODUK):
28             return produk
29         with open(FILE_PRODUK, "r") as f:
30             for baris in f:
31                 data = baris.strip().split("|")
32                 if len(data) == 3:
33                     kode, nama, harga = data
34                     produk[kode] = {"nama": nama, "harga": int(harga)}
35         return produk
36
37     def simpan_produk(produk):
38         with open(FILE_PRODUK, "w") as f:
39             for kode, info in produk.items():
40                 f.write(f"{kode}|{info['nama']}|{info['harga']}\n")
41
42     def tambah_produk(produk):
43         print("\n--- Tambah Produk ---")
44         kode = input("Kode produk   : ").upper()
45         if kode in produk:
46             print(f"✗ Kode {kode} sudah ada!")
47             return
48         nama = input("Nama produk   : ")
49         harga = input_angka("Harga       : Rp ")
50         produk[kode] = {"nama": nama, "harga": harga}
51         simpan_produk(produk)
52         print(f"✓ Produk '{nama}' ditambahkan")
53
54     def tampil_produk(produk):
55         if len(produk) == 0:
56             print("\n(Belum ada produk)")
57             return
58         print("\n" + "=" * 55)
59         print(f"{'Kode':<8}{'Nama Produk':<30}{'Harga':>15}")
60         print("=" * 55)
61         for kode, info in produk.items():
62             print(f"{kode:<8}{info['nama']:<30}{format_rupiah(info['harga']):>15}")
63         print("=" * 55)
64
65     # ===== TRANSAKSI =====
66     def transaksi(produk):
67         if len(produk) == 0:
68             print("\n✗ Tambahkan produk terlebih dahulu!")
69             return
70

```

```

71     keranjang = []
72     print("\n===== TRANSAKSI BARU =====")
73     tampil_produk(produk)
74
75     while True:
76         kode = input("\nKode produk (X=selesai): ").upper()
77         if kode == "X":
78             break
79         if kode not in produk:
80             print(f"✗ Kode '{kode}' tidak ditemukan")
81             continue
82
83         jumlah = input_angka(f"Jumlah beli '{produk[kode]['nama']}': ")
84         if jumlah <= 0:
85             print("✗ Jumlah harus positif")
86             continue
87
88         subtotal = produk[kode]["harga"] * jumlah
89         keranjang.append({
90             "kode"      : kode,
91             "nama"      : produk[kode]["nama"],
92             "harga"     : produk[kode]["harga"],
93             "jumlah"    : jumlah,
94             "subtotal"  : subtotal,
95         })
96         print(f"✓ +{jumlah}x {produk[kode]['nama']} ({format_rupiah(subtotal)})")
97
98     if len(keranjang) == 0:
99         print("\n(Transaksi dibatalkan – keranjang kosong)")
100         return
101
102     proses_pembayaran(keranjang)
103
104 def proses_pembayaran(keranjang):
105     total = sum(item["subtotal"] for item in keranjang)
106     print("\n--- Ringkasan Belanja ---")
107     for item in keranjang:
108         print(f" {item['nama']} ({item['jumlah']}x) = {format_rupiah(item['subtotal'])}")
109     print(f" TOTAL: {format_rupiah(total)}")
110
111     # Diskon otomatis
112     diskon = 0
113     if total >= 500000:
114         diskon = total * 0.10
115         print(f" 🎁 Diskon 10% (belanja ≥500rb): -{format_rupiah(diskon)}")
116     elif total >= 200000:
117         diskon = total * 0.05
118         print(f" 🎁 Diskon 5% (belanja ≥200rb): -{format_rupiah(diskon)}")
119
120     total_bayar = total - diskon
121

```

```

122     while True:
123         try:
124             bayar = int(input(f"\nTotal tagihan {format_rupiah(total_bayar)}. Uang
diterima: Rp "))
125             if bayar < total_bayar:
126                 print(f"✗ Uang kurang Rp {total_bayar - bayar:,}")
127                 continue
128             break
129         except ValueError:
130             print("✗ Masukkan angka yang valid")
131
132     kembalian = total_bayar - bayar
133     cetak_struk(keranjang, total, diskon, total_bayar, bayar, kembalian)
134
135     # ===== STRUK =====
136     def cetak_struk(keranjang, total, diskon, total_bayar, bayar, kembalian):
137         pastikan_folder()
138         waktu = datetime.now()
139         id_trx = waktu.strftime("%Y%m%d%H%M%S")
140
141         garis = "-" * 42
142         baris = []
143         baris.append(garis)
144         baris.append("          TOKO KELONTONG BAROKAH")
145         baris.append("          Jl. Ahmad Yani Km.5 Banjarmasin")
146         baris.append(garis)
147         baris.append(f"No. Transaksi : TRX-{id_trx}")
148         baris.append(f"Tanggal       : {waktu.strftime('%d-%m-%Y %H:%M:%S')}")
149         baris.append("-" * 42)
150         baris.append(f"{'Nama':<20}{'Qty':>5}{'Subtotal':>17}")
151         baris.append("-" * 42)
152         for item in keranjang:
153             nama = item["nama"][:18]
154
155             baris.append(f"{'nama':<20}{'item['jumlah']':>5}{format_rupiah(item['subtotal']):>17}")
156             baris.append("-" * 42)
157             baris.append(f"{'Total':<25}{format_rupiah(total):>17}")
158             if diskon > 0:
159                 baris.append(f"{'Diskon':<25}{'-' + format_rupiah(diskon):>17}")
160                 baris.append(f"{'Total Bayar':<25}{format_rupiah(total_bayar):>17}")
161                 baris.append(f"{'Tunai':<25}{format_rupiah(bayar):>17}")
162                 baris.append(f"{'Kembalian':<25}{format_rupiah(kembalian):>17}")
163             baris.append(garis)
164             baris.append("          Terima kasih atas kunjungan Anda")
165             baris.append("          Selamat berbelanja kembali!")
166             baris.append(garis)
167
168         struk = "\n".join(baris)
169
170         # Cetak ke layar
171         print("\n" + struk)
172
173         # Simpan file struk

```

```

173     nama_struk = f"{FOLDER_STRUK}/struk_{id_trx}.txt"
174     with open(nama_struk, "w") as f:
175         f.write(struk)
176     print(f"\n📄 Struk tersimpan: {nama_struk}")
177
178     # Tambah ke riwayat
179     with open(FILE_RIWAYAT, "a") as f:
180         f.write(f"{id_trx}|{waktu.strftime('%d-%m-%Y %H:%M')}|{total_bayar}|{len(keranjang)}\n")
181
182     # ===== RIWAYAT =====
183     def tampil_riwayat():
184         if not os.path.exists(FILE_RIWAYAT):
185             print("\n(Belum ada transaksi)")
186             return
187         print("\n" + "=" * 65)
188         print(f"{'ID Transaksi':<20}{'Tanggal':<22}{'Item':>5}{'Total':>18}")
189         print("=" * 65)
190         total_semua = 0
191         jumlah_trx = 0
192         with open(FILE_RIWAYAT, "r") as f:
193             for baris in f:
194                 data = baris.strip().split("|")
195                 if len(data) == 4:
196                     id_trx, tgl, total, item = data
197                     print(f"TRX-
{id_trx:<16}{tgl:<22}{item:>5}{format_rupiah(int(total)):>18}")
198                     total_semua += int(total)
199                     jumlah_trx += 1
200         print("=" * 65)
201         print(f"Total Transaksi: {jumlah_trx} | Total Omset:
{format_rupiah(total_semua)}")
202
203     # ===== MENU UTAMA =====
204     def tampilkan_menu():
205         print("\n" + "=" * 45)
206         print("    APLIKASI KASIR - TOKO BAROKAH")
207         print("=" * 45)
208         print("1. Tambah Produk")
209         print("2. Lihat Daftar Produk")
210         print("3. Mulai Transaksi")
211         print("4. Lihat Riwayat Transaksi")
212         print("5. Keluar")
213         print("=" * 45)
214
215     def main():
216         produk = muat_produk()
217
218         while True:
219             tampilkan_menu()
220             pilih = input("Pilih menu [1-5]: ")
221
222             if pilih == "1": tambah_produk(produk)

```

```

223         elif pilih == "2": tampil_produk(produk)
224         elif pilih == "3": transaksi(produk)
225         elif pilih == "4": tampil_riwat()
226         elif pilih == "5":
227             print("\n✓ Terima kasih telah menggunakan aplikasi ini!")
228             break
229         else:
230             print("✗ Pilihan tidak valid!")
231
232     if __name__ == "__main__":
233         main()

```

8.3 Contoh Hasil Eksekusi

Menambah Produk:

```

=====
      APLIKASI KASIR - TOKO BAROKAH
=====
1. Tambah Produk
2. Lihat Daftar Produk
3. Mulai Transaksi
4. Lihat Riwayat Transaksi
5. Keluar
=====
Pilih menu [1-5]: 1

--- Tambah Produk ---
Kode produk   : B001
Nama produk   : Beras Premium 5kg
Harga         : Rp 75000
✓ Produk 'Beras Premium 5kg' ditambahkan

```

Melihat Daftar Produk:

```

Pilih menu [1-5]: 2

=====
Kode   Nama Produk                               Harga
=====
B001   Beras Premium 5kg                        Rp 75.000
M001   Minyak Goreng 2L                        Rp 32.000
G001   Gula Pasir 1kg                          Rp 14.000
T001   Telur 1kg                               Rp 28.000
K001   Kopi Sachet                             Rp 2.500
=====

```

Proses Transaksi dengan Cetak Struk:

```

Pilih menu [1-5]: 3

===== TRANSAKSI BARU =====

Kode produk (X=selesai): B001
Jumlah beli 'Beras Premium 5kg': 2
✓ +2x Beras Premium 5kg (Rp 150.000)

Kode produk (X=selesai): M001
Jumlah beli 'Minyak Goreng 2L': 3
✓ +3x Minyak Goreng 2L (Rp 96.000)

Kode produk (X=selesai): T001
Jumlah beli 'Telur 1kg': 1
✓ +1x Telur 1kg (Rp 28.000)

Kode produk (X=selesai): X

--- Ringkasan Belanja ---
  Beras Premium 5kg (2x) = Rp 150.000
  Minyak Goreng 2L (3x) = Rp 96.000
  Telur 1kg (1x) = Rp 28.000
  TOTAL: Rp 274.000
  🛍 Diskon 5% (belanja ≥200rb): -Rp 13.700

Total tagihan Rp 260.300. Uang diterima: Rp 300000

=====
          TOKO KELONTONG BAROKAH
        Jl. Ahmad Yani Km.5 Banjarmasin
=====
No. Transaksi : TRX-20260423103045
Tanggal       : 23-04-2026 10:30:45
-----
Nama           Qty           Subtotal
-----
Beras Premium 5kg      2      Rp 150.000
Minyak Goreng 2L       3      Rp 96.000
Telur 1kg             1      Rp 28.000
-----
Total                               Rp 274.000
Diskon                             -Rp 13.700
Total Bayar                               Rp 260.300
Tunai                               Rp 300.000
Kembalian                               Rp 39.700
=====
          Terima kasih atas kunjungan Anda
          Selamat berbelanja kembali!

```

```
=====
📄 Struk tersimpan: struk/struk_20260423103045.txt
```

Melihat Riwayat Transaksi:

```
Pilih menu [1-5]: 4

=====
ID Transaksi      Tanggal           Item      Total
=====
TRX-20260423103045  23-04-2026 10:30      3      Rp 260.300
TRX-20260423104512  23-04-2026 10:45      5      Rp 425.800
TRX-20260423110233  23-04-2026 11:02      2      Rp 85.000
=====
Total Transaksi: 3 | Total Omset: Rp 771.100
```

8.4 Penjelasan Struktur Program

Program di atas mendemonstrasikan integrasi berbagai konsep yang dipelajari pada bab-bab sebelumnya. Struktur data Dictionary digunakan untuk menyimpan produk dengan kode sebagai key. List digunakan untuk keranjang belanja. Fungsi-fungsi dipisah berdasarkan tanggung jawabnya, misalnya tambah_produk, transaksi, dan cetak_struk.

Pengolahan file diterapkan pada tiga tempat: file produk.txt menyimpan daftar produk, file riwayat_transaksi.txt mencatat semua transaksi, dan folder struk/ berisi file struk untuk setiap transaksi. Error handling diterapkan melalui fungsi input_angka yang memvalidasi input user serta blok try-except pada operasi file. Percabangan dan perulangan digunakan pada hampir seluruh bagian program untuk mengontrol alur eksekusi.

8.5 Pengembangan Lanjutan

Setelah memahami program ini, mahasiswa ditantang untuk mengembangkan fitur tambahan seperti: manajemen stok barang (otomatis berkurang saat terjual), laporan penjualan harian/bulanan, sistem member dengan poin, kategori produk, fitur edit dan hapus produk, serta export laporan ke format CSV untuk analisis lebih lanjut.

💡 **Catatan:** Mini project ini adalah titik awal. Teruslah mengembangkan, menambah fitur, dan memperbaiki kode. Membaca kode orang lain dan mencoba memodifikasinya adalah cara terbaik untuk berkembang sebagai programmer.

DAFTAR PUSTAKA

- [1] Lutz, M. (2023). Learning Python, 6th Edition. O'Reilly Media.
- [2] Matthes, E. (2023). Python Crash Course, 3rd Edition. No Starch Press.
- [3] McKinney, W. (2022). Python for Data Analysis, 3rd Edition. O'Reilly Media.
- [4] Ramalho, L. (2022). Fluent Python, 2nd Edition. O'Reilly Media.
- [5] Python Software Foundation. (2026). Python 3 Documentation. <https://docs.python.org/3/>
- [6] Visual Studio Code Team. (2026). VS Code Python Tutorial. <https://code.visualstudio.com/docs/python/python-tutorial>
- [7] Munir, R. (2021). Algoritma dan Pemrograman dalam Bahasa Pascal, C, dan C++. Informatika Bandung.
- [8] Sitorus, L. (2020). Algoritma dan Pemrograman. Penerbit ANDI.